CSB-01 · REAL-TIME DETECTION OF CROSS-CHAIN BRIDGE EXPLOITS · CHAINALYSIS

BridgeWitness

Real-time detection of cross-chain bridge exploits, accusing only with proof.

Signatures can be stolen. Conservation can't.

Prototype specification · locked 9 October 2026
4 unique ideas · 3 ML models · 6 algorithms · real Nomad & Ronin data · live bridge tail
Backend: Docker on Render (< 512 MB) · Frontend: Vercel

Contents

1. The problem in plain words
2. What the problem statement asks for
3. Our solution in one paragraph
4. Our four unique ideas (USPs)
5. Other features
6. How someone uses it (user flow)
7. The three ML models
8. The six algorithms
9. How we avoid false accusations
10. Closing the loopholes (threat model)
11. Impact and benefits
12. Why we stand out
13. Technical depth
14. Tech stack
15. Technical architecture
16. Deployment: Render + Vercel
17. Data and API keys
18. Feasibility and viability
19. Honesty rules
20. Research we build on
21. Glossary

---

1. The problem in plain words

A **cross-chain bridge** lets people move money from one blockchain to another. You lock coins on chain A, and the bridge releases the same amount on chain B. The bridge is a vault holding everyone's locked coins, so it is the biggest target in crypto.

- Chainalysis estimated about **$2 billion was stolen from bridges in 2022**, roughly two thirds of all crypto stolen that year.
- **Ronin (March 2022, ~$624M public estimate):** attackers stole 5 of the 9 validator keys and signed two withdrawals **115 seconds apart**. Every signature was valid, so normal checks saw nothing wrong.
- **Nomad (August 2022, ~$190M):** a bug let anyone withdraw money that was never deposited. One attacker went first, then **hundreds of copycats** pasted the same transaction with their own address. In our dataset, 382 releases came from 279 addresses in **2.5 hours**.

The core issue: exploits finish in minutes, and today's tools either alert too late, alert too often (people start ignoring them), or can't explain *why* something is an attack.

2. What the problem statement asks for

CSB-01 asks for	Where BridgeWitness does it
A monitoring dashboard	Live console with a real-time flow graph
A live or replayed transaction feed from a public bridge dataset	Real Nomad and Ronin data replayed at 1–1000× speed, plus a **live tail** of a real bridge (Across)
Flag **volume spikes**	**Contagion Radar** (USP 2)
Flag **unusual signer behaviour**	**Signer DNA** from real recovered signatures (USP 3)
Flag **mismatched deposit/withdrawal pairs**	**Ledger Pairing** (USP 4)
A **risk-scored alert**	Risk 0–100 with an error-controlled alert gate, the **Accusation Budget** (USP 1)
The **suspected exploit type**	Exploit-type classifier with probability, reasons and the closest past incident
A working demo on a simulated attack replay	Seeded simulated attack replay, plus the two real historic attacks

3. Our solution in one paragraph

BridgeWitness watches both sides of a bridge at once. For every coin released on the destination chain it asks: **was a matching coin locked on the source chain?** That question (conservation) cannot be faked with stolen keys. Around it sit three smart signals: how fast an attack is *spreading* (copycats), *who actually signed* the withdrawal, and whether each deposit/withdrawal pair *looks normal for that route*. Every signal is passed through a statistical gate that **limits the share of false alarms**. Only a broken conservation rule, backed by evidence anyone can re-check, is allowed to call something an **exploit**. The result shows up live as a graph, with a plain-English explanation and a "what if we had stopped it here?" figure.

---

4. Our four unique ideas (USPs)

USP 1 · Accusation Budget: "We only accuse with proof, and we measure our false-alarm rate"

The problem it solves: monitoring tools fire alerts from fixed thresholds ("more than $1M", "3× normal volume"). Nobody can say how many of those alerts are false, so teams get alert fatigue and miss the real one.

What we do, in plain words:

1. Each signal gives a **p-value**: "how surprising is this compared with normal traffic?" It is calibrated on real normal history (*conformal p-value*).
2. We turn it into an **e-value** (a betting score) and pass it through **e-LOND**, a modern statistical method that keeps the expected share of false alerts **at or below 5%**, even when signals are correlated.
3. A live **budget meter** shows how much of that 5% has been spent.

4. Two levels of alert:
    - **Suspicious** (risk up to 69): statistics say it's unusual. Advice only.
    - **Exploit** (risk 70–100): also requires the **conservation rule to break**, with an evidence bundle anyone can re-verify in their browser.

Why it's unique: none of the tools we surveyed (research tools, Forta bots, hackathon projects) puts a statistical error guarantee on its alerts.

USP 2 · Contagion Radar: "The R_0 of an exploit"

The problem it solves: volume-spike detectors look at *size*. Nomad was drained by *spread*: hundreds of small copy-paste withdrawals from new addresses.

What we do:

1. **Fingerprint** every withdrawal (token, amount range, reused message id, batch shape, with the recipient address masked) and group near-identical ones with **MinHash-LSH**. That gives copycat clusters.
2. Score sudden bursts of new token→recipient links with **MIDAS-R**, a streaming graph burst detector.
3. Fit a **Hawkes process**, a model where each event can trigger more events, as in epidemics. Its **branching ratio $\hat{n}$** is the attack's "reproduction number":
    - **$\hat{n} < 1$:** dying out.
    - **$\hat{n} > 1$:** self-sustaining swarm. Hold the route now.

Why it's unique: nobody measures how fast an exploit spreads. This turns "volume spike" into "this attack is going viral, act now".

USP 3 · Signer DNA: "Who actually signed, recovered from the blockchain"

The problem it solves: "unusual signer behaviour" is usually simulated in demos. Real bridges hide signers inside transaction data.

What we do:

1. Each Ronin withdrawal carries the validators' signatures inside its transaction data. We **mathematically recover** (`ecrecover`) the exact validators who signed every withdrawal.
2. We build each validator's normal behaviour: how often they sign, who they usually sign with (co-signing graph), at what hours, and for what values.
3. An ML model scores each new **quorum** (the group of signers) for how unusual it is. It also checks the **withdrawal-ID jump**: the attack used IDs 2,000,000 and 2,000,001 while normal IDs were around 1.0 million, which means the sidechain never issued those IDs.

Already verified: we recovered the 5 validators that signed both Ronin attack withdrawals from public data. They are exactly 5 of the 9 registered validators, matching the public post-mortem.

Measured on 4,200 real normal withdrawals (Jan–Mar 2022, signers recovered from chain):

- The attack's 5-validator group also signed **276 normal withdrawals (6.6%)**, and four of those validators sign almost every withdrawal. So *who* signed was **not** unusual on its own. This is exactly why Ronin's checks saw nothing wrong, and we say so instead of claiming a false signal.

- What **was** unusual: the withdrawal IDs. Normal IDs ran from 1,029,086 to 1,064,238; the attack used **2,000,000 and 2,000,001**, about 936,000 beyond any ID the sidechain had issued. Signer DNA scores this ID jump, plus the value signed against each group's history, as its main evidence.

Why it's unique: other projects simulate signers. Ours are real, recovered from chain data, and the result is reported honestly.

USP 4 · Ledger Pairing: "Every coin out must have a coin in"

The problem it solves: most tools match deposits to withdrawals by message id. Nomad copycats **reused real message ids** (one id was used 279 times), so id-matching is fooled.

What we do:

1. An ML model learns, for each route, the **normal time window** between a deposit and its withdrawal. *Conformal* calibration guarantees a coverage level (e.g. 99%).
2. The **Hungarian algorithm** finds the best one-to-one matching of deposits to withdrawals, using amount, recipient and timing, not just the id.
3. A withdrawal with no deposit inside its window is a **mismatched pair**.
4. A per-token **conservation ledger** (coins locked minus coins released) is drawn live. **Click the gap** to see the exact transactions causing it.

Why it's unique: totals-only monitors know *that* money is missing but not *which* transactions took it. Id-join tools can be fooled. We do both, live.

---

5. Other features

Feature	What it does
Exploit-type label	Every alert names the suspected attack (fake deposit, replay, key compromise, copycat swarm, amount mismatch) with a probability and the top 3 reasons
Precedent match	"This looks most like: Nomad 2022", the closest past incident
Live Flow Graph	Moving network of chains → bridge → tokens → recipients; colours show the verdict; copycat clusters are outlined; validators ring the bridge
Baseline scorecard	The same replay run through the common methods (whale threshold, volume z-score, id matching) next to ours: time to first alert, money lost before the alert, false alerts per day
What-if (counterfactual)	"If the route had been held at the first alert, $X would not have left", computed from the data
Verify in browser	The evidence for an exploit alert is re-checked in the user's own browser, so they don't have to trust our server
Live tail	Real, current Across bridge traffic through the same pipeline, showing normal pairs being confirmed with no false alarms

Feature	What it does
Lifecycle states	pending → consistent / overdue / mismatched; `unknown` when data is missing, never a false accusation

6. How someone uses it (user flow)

A judge or analyst understands the product in about 60 seconds:

1. **Pick a feed:** Nomad 2022, Ronin 2022, simulated attack, or live.
2. **Press play:** the flow graph fills in real time; normal pairs turn green.
3. **The alert fires:** a card slides in showing risk, exploit type, probability and the closest past incident.
4. **Click "Why":** plain-English reasons, the signals with their scores, and the exact culprit transactions.
5. **Click "Verify":** the evidence is re-checked in the browser.
6. **Click "What if we held here?":** money that would have been saved.
7. **Scorecard tab:** how BridgeWitness compares with the usual methods on the same data.

7. The three ML models

	Model	What it does in plain words	Data it learns from	How it is trained	What it outputs
ML1	**Pairing-delay model**: LightGBM quantile regression + conformal calibration (CQR)	Learns how long a normal deposit→withdrawal takes on each route, so a withdrawal arriving with no fitting deposit stands out	4,482 real Nomad pairs + 22,830 real Ronin pairs	Time-ordered 60/20/20 split (train/calibrate/test); the calibration step guarantees the window covers ~99% of normal pairs	A time window `[earliest, latest]` per withdrawal and its measured coverage
ML2	**Signer anomaly model**: Isolation Forest + Dirichlet–multinomial quorum likelihood	Learns which validator groups normally sign together and how; scores how unusual a new signing group is	Real Ronin withdrawals with signers recovered from chain data (~4,000 sampled, Jan–Mar 2022)	Fit on Jan–Feb, calibrate on March before the hack, test on the attack	Quorum "surprise" score and a calibrated p-value
ML3	**Exploit-type classifier**: LightGBM multiclass + isotonic calibration + TreeSHAP reasons	Given an alert's signals, names the most likely attack type and explains why	Seeded simulated attacks mixed into real background traffic	5-fold cross-validation on simulated data; **tested on the real Nomad and Ronin incidents**, which it never saw	Exploit type, probability, top-3 reasons, closest past incident

Notes:

- TreeSHAP reasons come from LightGBM's built-in contribution output (`pred_contrib`), which is the same exact algorithm without the heavy `shap` library. This keeps the server under 512 MB.
- ML can rank and explain, but **ML alone never labels an exploit** (see §9).

8. The six algorithms

	Algorithm	What it does in plain words	Used for
A1	**Conservation ledger** (double-entry invariant)	Keeps a running total per token: coins released must never exceed coins locked. If they do, something was released without backing	The only thing allowed to say "exploit"
A2	**Hungarian matching**	Finds the best one-to-one pairing of deposits and withdrawals, so reused ids can't fool it	Mismatched pairs (USP 4)
A3	**MinHash-LSH**	Quickly finds near-identical transactions among thousands, like spotting copy-pasted homework	Copycat clusters (USP 2)
A4	**MIDAS-R**	Spots sudden bursts of new links in a streaming graph, using tiny fixed memory	Burst score (USP 2)
A5	**Hawkes process** (self-exciting)	Measures whether events are triggering more events; the branching ratio says whether the attack is spreading	Exploit R_0 (USP 2)
A6	**Conformal p-values → e-values → e-LOND**	Turns any score into a calibrated "how surprising" number and keeps the share of false alerts at or below 5% over the whole stream	Accusation Budget (USP 1)

Risk score: `risk = 100 × (1 − combined p-value)` , capped at **69** unless the conservation rule (A1) breaks. So an alert above 69 always has hard evidence behind it.

---

9. How we avoid false accusations

1. **Rules decide, signals advise.** Only a broken conservation rule with evidence can produce "exploit". Statistics and ML can only produce "suspicious".
2. **Measured error rate.** e-LOND controls the false-alert share; we also **measure** it on held-out normal days and show the real number.
3. **"Unknown" is a valid answer.** If one chain's data is late or missing, the status is `unknown` , never `violation` .
4. **Only final data counts.** Unconfirmed blocks don't enter the ledger, so reorganisations can't create fake gaps.
5. **Models abstain on new routes.** Without enough normal history, a model says nothing instead of guessing.
6. **Evidence anyone can check.** Exploit alerts carry a bundle that re-verifies in the browser.

10. Closing the loopholes (threat model)

How an attacker might try to hide	What catches it
Split the theft into many small withdrawals below thresholds	Conservation ledger (sums everything) + Contagion Radar
Drain slowly over days	The ledger gap only grows; e-LOND keeps watching over time
Use a fresh address for every withdrawal	Fingerprints ignore the address; they match the transaction's shape
Use stolen keys at normal times	Signer DNA scores *who signed together*, not just *when*
Reuse a real, legitimate message id (as in Nomad)	Pairing uses amount, recipient and time, not only the id
Exploit a slow or broken data feed	Status becomes `unknown`; per-chain watermarks show feed health
Chain reorganisation	Only finalized blocks count
Poison the "normal" history	Calibration uses only past, finalized data; model files are hash-pinned
Manipulate token prices	The rule works in token units; dollars are for display only

11. Impact and benefits

Who	Benefit
Bridge operators	Early warning with a reason and a recommended action (hold the route) while an attack is still spreading
Investigators (e.g. Chainalysis)	Culprit transactions, signer evidence and exploit type in one place, from minute one
Exchanges and custodians	Know which incoming bridged funds are unbacked and should be frozen or flagged
Ordinary users	Fewer drained bridges and fewer false pauses that freeze their money

What we measure and show (from our own scripts, not guesses):

- Time from the first malicious release to our first alert, per incident.
- Money that left before the alert, versus after.
- False alerts per normal day, ours against common baselines.
- "Money that a hold at the first alert would have kept in the bridge" (counterfactual).

Real stakes: Nomad's copycat swarm took 2.5 hours. Detecting it as spreading early, and holding the route, protects most of what left later.

12. Why we stand out

	Typical hackathon project	Research tools (XChainWatcher, BridgeShield)	Industry bots (Forta)	BridgeWitness
Real-time	Simulated	Offline	Yes, one chain per bot	**Yes, both chains, live graph**
Mismatched pairs	id join	Rules / graph model	No	**Learned windows + optimal matching + culprit drill-down**
Signer behaviour	Simulated	No	No	**Real signers recovered from chain**
Volume spikes	z-score	No	z-score	**Spread measurement (exploit R_0)**
False-alarm control	None	None	None	**Statistical guarantee + measured rate**
Exploit type	LLM guess	Coarse (src/dst/off-chain)	No	**Calibrated type + reasons + precedent**
Evidence	None	Paper	None	**Re-verifiable in the browser**

13. Technical depth

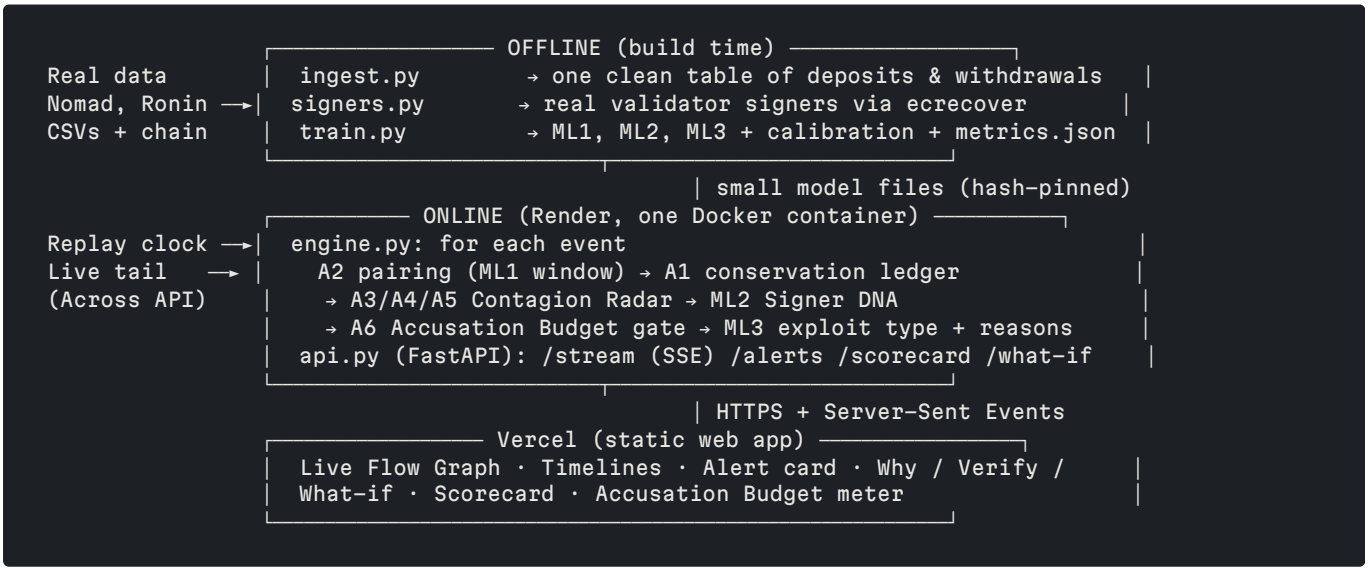
- **Statistics:** conformal prediction (distribution-free guarantees), e-values, online FDR under arbitrary dependence (e-LOND).
- **Streaming algorithms:** count-min sketches (MIDAS-R), locality-sensitive hashing (MinHash), point processes (Hawkes MLE).
- **Optimisation:** Hungarian algorithm for optimal assignment.
- **Cryptography / blockchain:** ABI decoding of raw calldata, EIP-191 signed-message hashing, secp256k1 public-key recovery (`ecrecover`), validator-set checks, finality handling.
- **ML engineering:** quantile gradient boosting, isolation forests, calibrated multiclass boosting, exact TreeSHAP explanations, time-ordered splits with no leakage, held-out real incidents.
- **Systems:** event-time replay engine, Server-Sent Events streaming, WebGL graph rendering, a container under 512 MB.

14. Tech stack

Layer	Technology	What it does
Backend language	**Python 3.10**	All models, algorithms and the replay engine
API server	**FastAPI + Uvicorn**	REST endpoints and the live event stream (SSE)

Layer	Technology	What it does
Data	**pandas, NumPy, PyArrow**	Loading, joining and storing bridge data
Maths	**SciPy**	Hungarian algorithm, Hawkes likelihood optimisation
ML	**LightGBM**	Quantile (ML1) and multiclass (ML3) models, built-in TreeSHAP
ML	**scikit-learn**	Isolation Forest (ML2), isotonic calibration
Similarity	**datasketch**	MinHash-LSH copycat clustering
Blockchain	**eth-keys, eth-utils**	Signature recovery, keccak hashing
Chain access	**Public Ethereum RPC** (drpc, keyless)	Reading historic withdrawal transactions
Frontend	**React 18 + Vite + TypeScript**	The web app
Graph	**Sigma.js + Graphology + ForceAtlas2**	Fast WebGL live flow graph
Charts	**uPlot**	Small, fast real-time timelines
Design	**Oxide Vault** design system	Consistent, readable, colour-meaningful UI
Backend hosting	**Render** (Docker)	Runs the API container
Frontend hosting	**Vercel**	Serves the web app globally

15. Technical architecture



Flow of one event: a withdrawal arrives → we look for its deposit inside the learned window → update the ledger → update the copycat, burst and spread signals → check the signers → the statistics gate decides "nothing", "suspicious" or (with ledger proof) "exploit" → the classifier names the type → the browser draws it.

16. Deployment: Render + Vercel

Part	Where	How
Backend API	**Render** web service, **Docker image** (built from `D:\fusionhack\bridgewitness`)	`python:3.10-slim` base, one Uvicorn worker, models loaded once at start
Frontend	**Vercel**	Static Vite build; `VITE_BW_API` points to the Render URL

Staying under Render's 512 MB memory limit:

- Training happens **offline**. The container only loads small model files; no training at runtime.
- **No PyTorch, no `shap` library:** reasons use LightGBM's built-in TreeSHAP.
- Replays are **pre-computed** into compact files and streamed. We don't re-run heavy joins per visitor.
- One worker process; data kept as NumPy arrays and only the needed columns.
- Expected memory around 250–300 MB (Python + NumPy + pandas + SciPy + scikit-learn + LightGBM). Checked with `docker stats` before deploying.
- Free Render services sleep when idle. The web app shows a "waking the server" state, and the replays also exist as static files so the demo never breaks.

CORS: the API allows only the Vercel domain. There are no secrets in the browser.

17. Data and API keys

Data	Source	Key needed?
Nomad and Ronin bridge transfers (normal + attack), labelled	XChainWatcher public dataset (pinned, checksummed CSVs)	No
Ronin validator signatures	Public Ethereum transactions via `https://eth.drpc.org`	No
Live bridge traffic	Across public API `app.across.to/api/deposits`	No
Token prices (live tail)	DefiLlama `coins.llama.fi`	No
Faster bulk chain reads (optional)	Etherscan API (free tier)	Optional `ETHERSCAN_API_KEY`

Result: the whole prototype runs with zero paid services and zero required API keys.

18. Feasibility and viability

Feasibility (can it be built?):

- Build window: **3.5 hours**, on a vertical-slice-first plan, so a working demo exists after the first hour and every later step only adds to it.

- All libraries are already installed. All data is already downloaded or reachable without keys.
- The riskiest part (recovering real signers) is **already proven** on the Ronin attack transactions.
- Training all three models takes minutes on a laptop.

Viability (does it make sense beyond the hackathon?):

- **Cost:** free tiers (Render + Vercel + public RPC) for the prototype. In production, one small server per bridge.
- **Who pays:** bridge operators (protection and insurance requirements), exchanges and custodians (deposit screening), investigation firms (faster incident response).
- **Scales:** each signal is constant-time per event (sketches, hashing, incremental ledger), and new bridges only need a small adapter that reads their deposit and withdrawal events.
- **Fits the ecosystem:** works alongside pre-execution tools and forensics. It is the missing "is this release actually backed?" layer between them.

19. Honesty rules

- We never say "100% accurate", "AI-powered" or "funds saved". We say "would have kept in the bridge, computed from the data".
- Results are reported **per incident**, not as one global accuracy figure, because there are only a few real attacks.
- Dataset labels come from XChainWatcher's rules; we say so.
- Simulated data is always labelled as simulated.
- The 5% false-alarm guarantee is stated with its assumptions, next to the **measured** rate.
- Ronin: $624M is the public estimate; the dataset value is $565.4M. We label both.
- If Signer DNA finds the attack quorum was common, we show that honestly and rely on the withdrawal-ID and value signals.

20. Research we build on

Work	What we take from it
Count of Monte Crypto: Accounting-based Defenses for Cross-Chain Bridges (Liu et al., UCSD, arXiv 2410.01107)	The conservation (double-entry) invariant
XChainWatcher (Augusto et al., arXiv 2410.02029)	Cross-chain rules and the labelled Nomad/Ronin dataset
SoK: Not Quite Water Under the Bridge (Lee et al., arXiv 2210.16209); *SoK: Cross-Chain Bridging Design Flaws* (Notland et al., arXiv 2403.00405)	Attack taxonomy for exploit types
Conformalized Quantile Regression (Romano, Patterson, Candès, NeurIPS 2019)	ML1 guaranteed windows
Testing for outliers with conformal p-values (Bates et al., Annals of Statistics 2023)	Calibrated p-values for every signal
Online multiple testing with e-values (Xu & Ramdas, AISTATS 2024)	e-LOND false-alarm control

Work	What we take from it
MIDAS: Microcluster-Based Detector of Anomalies in Edge Streams (Bhatia et al., AAAI 2020)	Streaming burst detection
Hawkes (1971), self-exciting point processes	Exploit spread (R_0)
Broder (1997), MinHash	Copycat fingerprints
The Blockchain Imitation Game (Qin et al., USENIX Security 2023)	Evidence that copycat attacks are real and automatic
From local explanations to global understanding (Lundberg et al., Nature MI 2020)	TreeSHAP reasons
Compared against: BridgeShield (arXiv 2508.20517), BridgeGuard (WWW 2025), Forta bridge kit, XChainDataGen (arXiv 2503.13637)	What exists, and the gaps we fill

21. Glossary

Term	Meaning
Bridge	A system that moves value between blockchains by locking on one and releasing on the other
Deposit / withdrawal	Locking coins on the source chain / releasing them on the destination chain
Conservation	Coins released must never exceed coins locked
Validator / signer	A key holder whose signature approves a withdrawal; a **quorum** is the group that signed
ecrecover	Maths that recovers who signed from a signature
p-value	How surprising something is if everything were normal (small = surprising)
e-value	A "betting score" against normal; large = evidence of a problem
FDR	False discovery rate: the share of alerts that are false
Conformal	A method that turns any model into one with a guaranteed error level
Hawkes branching ratio	Average number of new events each event triggers; above 1 means spreading
MinHash-LSH	A fast way to find near-duplicates
Hungarian algorithm	Finds the best one-to-one pairing between two lists
SSE	Server-Sent Events: the server pushes updates to the browser live